

Shopify Store Analytics Dashboard

Enterprise E-Commerce Intelligence, Filter Architecture & Calculation Manual

A full-stack, production-grade analytics platform and merchant control dashboard built with the modern MERN stack (React 19, Node.js, Express, MongoDB Atlas). Features high-performance MongoDB aggregation pipelines, multi-period date range filtering, real-time debounced search, dynamic multi-style data visualizations, catalog velocity tracking, and complete Indian Rupee (INR) currency standardization.

Project Name	Shopify Store Analytics Studio
Store Name / Owner	Apex Retailers (Devicharan Prajapati)
Core Architecture	MERN Stack (React 19, Express.js, Node.js, MongoDB Atlas)
State Management	Redux Toolkit (@reduxjs/toolkit, react-redux)
Visualization Suite	Recharts (Dynamic Area, Dual Bar, Horizontal Bar, Line, Donut)
Styling & Theme	Tailwind CSS v4 (Shopify Clean White & Emerald #10B981)
Base Currency	Indian Rupee (INR / Rs.) with en-IN numbering format
Filter Engine	Date Range Presets & Custom, Debounced Live Search, Category, Status
GitHub Repository	https://github.com/DevicharanPrajapati/ShopifyAnalyticsDashboard.git
Build Release	v1.3.0 Production Master Release

Key Architecture Takeaway: Designed strictly as a single-merchant operational cockpit for Apex Retailers, this project eliminates e-commerce guesswork by marrying MongoDB Atlas multi-stage aggregation pipelines with client-side reactive charts. It delivers multi-range financial modeling, real-time live search, inventory stock velocity tracking, and comparative period-over-period growth metrics with sub-second response times.

1. Executive Summary & System Architecture

Modern e-commerce merchants require instant, multi-faceted visibility into transaction volume, shopper behavior, and inventory velocity. The **Shopify Store Analytics Dashboard** provides a responsive, robust control tower that converts raw transactional data into high-value financial intelligence.

Decoupled Multi-Tier Architecture

- **1. Client Layer (SPA):** Built on React 19 and Vite. Utilizes Redux Toolkit for centralized analytics and UI collapse state caching, Recharts for responsive SVG visualizations, and Tailwind CSS v4 for clean emerald styling.
- **2. API & Routing Layer:** Node.js and Express.js REST API with standardized request wrappers (asyncHandler), custom response contracts (ApiResponse), and centralized error handling (ApiError).
- **3. Service & Analytics Engine:** Business logic powered by native MongoDB multi-stage aggregation pipelines (\$match, \$unwind, \$group, \$bucket, \$lookup, \$project).
- **4. Database & Storage Layer:** MongoDB Atlas cloud cluster with compound indexes optimized for date-bounded queries across orders, products, and daily visitor footfall.

Complete Technology Stack

Domain	Technology	Role & Purpose in Project
Frontend Core	React 19 + Vite	High-speed reactive SPA with instant HMR and optimized bundle splitting
Global State	Redux Toolkit	Centralized date filtering, overview metrics, and UI collapse persistence
Visualizations	Recharts	Area charts, dual bar charts, horizontal bars, lines, and donut distributions
Styling Engine	Tailwind CSS v4	Utility-first layout, custom emerald palette, and fully responsive breakpoints
Backend Runtime	Node.js (v22) + Express	Lightweight, asynchronous REST API serving JSON endpoints
Database	MongoDB Atlas (Mongoose)	Document storage with aggregation pipelines, indexes, and schema validation
HTTP Client	Axios	Configured instance with base URLs, timeouts, and automated serialization
Iconography	Lucide React	Consistent visual icons for navigation, KPI cards, and header toggles

2. Application Modules: Dashboard, Orders & Products

2.1 Store Performance Dashboard (/)

- **Shopify-Style Interactive KPI Cards:** Four primary metric cards displaying Total Revenue, Total Orders, Conversion Rate, and Average Order Value (AOV). Clicking ANY card selects it, applying a highlight border and automatically morphing the primary chart to plot that exact metric.
- **Primary Dynamic Chart & Style Switcher:** Synchronized with the selected card. Displays a 'Selected: ' badge and features an in-chart style toggle switching between **Area Chart** (smooth gradient), **Bar Chart** (discrete columns), and **Line Chart** (trend line & points) with localStorage persistence.
- **Traffic vs Orders Trend (Dual Bar Chart):** Daily store footfall (visitors) versus conversions, revealing buyer purchase intent across weekdays and weekends.
- **Category Sales Distribution (Horizontal Bar Chart):** Ranks category revenue contributions (Electronics, Apparel, Home & Kitchen, Accessories, Bags, Footwear).
- **Payment Settlement Breakdown (Donut Chart):** Visualizes paid, pending, and refunded transactions with an interactive center counter.
- **Top Selling Products Leaderboard:** Live ranking of top revenue-generating items with SKU codes, unit sales, and gross revenue in INR.
- **Recent Orders Feed:** Live transaction table showing order ID, customer details, payment status badges, and transaction timestamps.

2.2 Orders Analytics & Management (/orders)

- **Date Range Presets & KPIs:** Renders Orders in Range, Net Order Value (INR), Average Order Value (AOV), and Fulfilled Orders count with percentage fulfillment rate.
- **Order Basket Value Tiers (Bar Chart):** Uses MongoDB's \$bucket aggregation stage to classify orders into basket size tiers (Under Rs. 1,000; Rs. 1,000 - Rs. 5,000; Rs. 5,000 - Rs. 10,000; Above Rs. 10,000).
- **Daily AOV Trend (Line Chart):** Tracks customer purchasing power fluctuations day-by-day.
- **Debounced Multi-Field Search:** Real-time filtering across Order Number, Customer Name, Email, City, and Item titles/SKUs with 1-click instant clear.
- **Status Filter Pills:** Instant filtering between All Orders, Paid, Pending, and Refunded states.

2.3 Products Catalog & Stock Velocity (/products)

- **Inventory KPI Cards:** Catalog Items count, Units in Stock, Units Sold in period, and Top Category with sales volume and percentage share.
- **Category Sales Share (Donut Chart):** Renders category revenue split with interactive hover tooltips.
- **Inventory Stock vs Units Sold (Dual Bar Chart):** Compares shelf inventory against sales velocity, warning merchants of stockouts.
- **Catalog Grid & Margins:** Displays product imagery, SKU codes, stock levels, INR retail prices, and computed profit margin percentages.

3. Comprehensive Filter System & Query Logic

The application provides a cohesive, multi-layered filtering architecture operating seamlessly between the React client and MongoDB Atlas backend. All filters maintain URL/Redux synchronization and prevent unnecessary database queries.

3.1 Date Range Filter Engine (DateFilter.jsx & dateHelper.js)

- **Presets Supported:** today (00:00:00 to 23:59:59), yesterday (prior full day), 7d (past 7 days), 30d (past 30 days - default), 90d (past 90 days), 1y (past 365 days), and custom.
- **Comparative Previous Period Math:** To power period-over-period growth comparisons, dateHelper.js calculates duration $\Delta t = \text{endDate} - \text{startDate}$. The prior period is automatically computed as $\text{prevEndDate} = \text{startDate}$ and $\text{prevStartDate} = \text{startDate} - \Delta t$, guaranteeing an exact apple-to-apple comparison window.
- **UTC Time Normalization:** Presets use `startOfDay()` and `endOfDay()` utilities to normalize timestamps into UTC boundaries, avoiding timezone-induced day clipping.

3.2 Real-Time Debounced Live Search Filter

- **300ms Client Debouncing:** An intelligent `useEffect` timer in `ProductsPage.jsx` and `OrdersPage.jsx` waits 300ms after the last keystroke before dispatching the API call, preventing request flooding while providing a live typing search experience.
- **Multi-Field Product Search:** Searches simultaneously across title, sku, and category using an unanchored, case-insensitive `$or` MongoDB regex query.
- **Multi-Field Order Search:** Searches simultaneously across `orderNumber`, `customer.name`, `customer.email`, `customer.city`, and order item title & sku.
- **1-Click Instant Clear:** An embedded X button appears inside the input whenever text exists, allowing instant clearing and view restoration. Active search chips display below the filter bar with result counts.

3.3 Filter Architecture Summary Table

Filter Type	Scope / Component	Client Logic	Backend MongoDB Pipeline
Date Range	Global Analytics	Redux <code>dateFilter</code> state with 6 presets & custom dates	<code>\$match: { orderDate: { \$gte: start, \$lte: end } }</code>
Live Search (Orders)	Orders Table	300ms debounced input, resets <code>page=1</code> , 1-click 'X'	<code>\$or: [orderNumber, customer.*, items.*]</code> with regex
Live Search (Products)	Catalog Grid	300ms debounced input, instant clear 'X', active filter chips	<code>\$or: [title, sku, category]</code> with regex
Category Filter	Products Catalog	Single-click category pills (Electronics, Apparel, Bags, etc.)	<code>\$match: { category: selectedCategory }</code>
Financial Status	Orders Management	Pill tabs: All, Paid, Pending, Refunded; resets <code>page=1</code>	<code>\$match: { financialStatus: status }</code>
Metric Morph Selector	Dashboard Chart	Clicking KPI card updates <code>selectedMetric</code> ; selected badge shown	Combines revenue Trend & traffic Trend into single series
Chart Style Switcher	RevenueChart	Toggles Area, Bar, Line; persists in <code>localStorage</code>	Client-side dynamic <code>Recharts</code> component switching

4. Analytical Calculations & Mathematical Formulas

Every KPI, chart trend, and financial statistic in the platform is computed strictly through mathematical algorithms equipped with defensive zero-division guards and standardized Indian numbering rules.

4.1 Core Financial Formulas & Guard Rules

Metric Name	Mathematical Formula	Zero-Division & Edge-Case Handling	Formatting & Units
Total Revenue	Revenue = Σ (order.totalAmount) for all paid & pending orders	If no orders exist in period, returns 0. Refunded orders are tracked separately.	INR Currency (₹) e.g. ₹7,48,282
Total Orders	Orders = Σ 1 for all orders within date window	Counts every valid transaction document matched in the date range.	Integer count e.g. 138 orders
Average Order Value (AOV)	AOV = Total Revenue / Total Orders	If Total Orders == 0, returns 0 immediately, preventing DivideByZero / NaN.	INR Currency (₹) e.g. ₹5,422
Conversion Rate (%)	CR = (Total Orders / Total Visitors) × 100	If Total Visitors == 0, returns 0.00%. Joins order count with VisitorTraffic aggregate.	Percentage (%) e.g. 3.42%
Period Growth (% Change)	$\Delta\% = ((\text{Current} - \text{Previous}) / \text{Previous}) \times 100$	- Both 0: 0% - Prev 0 & Curr > 0: +100% - Prev 0 & Curr < 0: -100%	Signed % with arrows e.g. +14.8%, -3.2%
Category Sales Share (%)	Share = (Category Revenue / Total Revenue) × 100	MongoDB \$unwind items, \$lookup products, and compute relative percentage.	Percentage (%) e.g. 38.5%
Product Profit Margin (%)	Margin = $((\text{Price} - \text{CostPrice}) / \text{Price}) \times 100$	If costPrice is undefined or price == 0, returns N/A safely.	Percentage (%) e.g. 45% margin
Basket Value Tiers	Segmented by MongoDB \$bucket into: <₹1k, ₹1k-₹5k, ₹5k-₹10k, >₹10k	Assigns orders to default overflow bucket if totalAmount exceeds highest boundary.	Bar chart series with order count & volume
Fulfillment Rate (%)	FR = (Fulfilled Orders / Total Orders) × 100	If Total Orders == 0, returns 0%. Evaluates fulfillmentStatus == 'fulfilled'.	Percentage (%) e.g. 88.4%

4.2 Comparative Growth Calculation Implementation

In server/src/utlis/dateHelper.js, growth is implemented as a standalone, deterministic pure function:

```
export const calculatePercentageChange = (current = 0, previous = 0) => {
  if (previous === 0) return current > 0 ? 100 : 0;
  const change = ((current - previous) / previous) * 100;
  return Math.round(change * 10) / 10; // Rounded to 1 decimal place
};
```

5. Database Schemas, Indexing & Data Engineering

5.1 Mongoose Schemas & Field Design

Mongoose schemas are engineered to support low-latency analytical aggregations over large transaction sets:

Model	Key Fields & Subdocuments	Index Architecture
Order (orders)	orderNumber, customer: { name, email, city }, items: [{ product, title, price, quantity }], totalAmount, subtotal, tax, financialStatus, fulfillmentStatus, orderDate	- Compound: { storeId: 1, orderDate: 1, financialStatus: 1 } - Single: orderNumber: 1
Product (products)	title, sku (unique), category, price (INR), costPrice (INR), inventoryQuantity, status (active/draft), image	- Text: { title: 'text', category: 'text' } - Single: category: 1, storeId: 1
VisitorTraffic (visitortraffics)	storeId, date, visitorsCount, sessionsCount	Unique Compound: { storeId: 1, date: 1 }

5.2 Index Optimization for Analytical Aggregations

- Compound Index { storeId: 1, orderDate: 1 }:** Satisfies all date-bounded range queries in \$match without collection scans.
- Covered Queries:** Queries on financialStatus and orderDate execute directly in RAM indexes, keeping response times under 15ms.
- Sparse SKU Index:** Ensures product uniqueness and rapid exact-match lookups during search and order item mapping.

5.3 Synthetic Data Seeding Engine

A realistic seeding script (server/src/seeds/seedData.js) populates 30 consecutive days of orders and traffic. It generates authentic Indian customer personas (Mumbai, Bengaluru, Delhi), realistic market pricing in INR (Rs. 1,499 - Rs. 12,999), and weekend-adjusted footfall delivering a realistic 2.5% - 4.5% conversion rate.

6. Backend REST API Reference

All endpoints adhere to REST conventions, returning standard HTTP codes and JSON payloads encapsulated by ApiResponse:

Method	Route Path	Query Parameters	Description
GET	/api/health	None	Service health status and UTC timestamp
GET	/api/analytics/overview	preset, startDate, endDate	KPI cards with % comparative growth
GET	/api/analytics/revenue-trend	preset, startDate, endDate	Daily revenue and order volume series
GET	/api/analytics/orders-stats	preset, startDate, endDate	Order value tiers and daily AOV trend
GET	/api/analytics/products-stats	preset, startDate, endDate	Category shares and stock vs units sold
GET	/api/analytics/dashboard	preset, startDate, endDate	Consolidated payload of all dashboard widgets
GET	/api/orders	status, search, page, limit	Paginated order list with search filters
GET	/api/orders/:id	None (path parameter)	Single order details by MongoDB ObjectId
GET	/api/products	category, search, page, limit	Paginated catalog products with search filter
GET	/api/products/:id	None (path parameter)	Single product details by MongoDB ObjectId

Standardized JSON Envelopes

Success Envelope (ApiResponse)	Error Envelope (ApiError)
<pre>{ "statusCode": 200, "success": true, "message": "Complete dashboard analytics retrieved", "data": { "overview": { "current": { ... } }, "revenueTrend": [...], "categorySales": [...] } }</pre>	<pre>{ "statusCode": 404, "success": false, "message": "Order not found: invalid ID format", "errors": [] }</pre>

7. Architectural Patterns & UI/UX Design System

7.1 Architectural Patterns & Utilities

- **asyncHandler Wrapper (server/src/utills/asyncHandler.js):** Eliminated repetitive try/catch blocks across every controller action. Automatically catches rejected promises and pipes them to Express's next() handler.
- **Centralized dateHelper (server/src/utills/dateHelper.js):** Provides startOfDay(), endOfDay(), calculatePercentageChange(), and getDateRangeFromQuery(). Standardizes time zone parsing and comparative period generation into a single call.
- **Defensive Input Validation:** Integrated mongoose.isValidObjectId() guards to eliminate unhandled CastErrors when querying by ID, gracefully returning HTTP 404 instead of HTTP 500 crashes.
- **Safe Pagination Clamping:** Clamps user pagination parameters with upper and lower bounds (Math.max(1, page) and Math.min(100, limit)), protecting the database from malicious unbounded queries.
- **Mongoose Lean Read Operations:** Employs .lean() on read-only queries to bypass full Mongoose document instantiation, drastically reducing memory overhead and JSON serialization latency.

7.2 UI/UX Design System & Navigation

- **Clean Shopify Emerald Theme:** Restored the crisp white and emerald green palette (#10B981) authentic to the Shopify merchant aesthetic, featuring clean slate backgrounds and subtle borders.
- **Indian Rupee (INR) Standardization:** All currency representations across cards, charts, and tables display formatted in Indian Rupees (₹ / Rs.) with Indian number system comma grouping (e.g. Rs. 7,48,282).
- **Stationary Fixed Desktop Sidebar:** Desktop sidebar is anchored with fixed positioning (fixed inset-y-0 left-0 w-64 z-30), eliminating scrolling misalignment during long page navigation.
- **Sidebar Hide/Show Collapse Mode:** Users can toggle the sidebar closed or open via the header icon or the standard Ctrl+B (Cmd+B) keyboard shortcut. When collapsed, the main content area smoothly transitions to full-width (lg:pl-0). State persists in localStorage.
- **Custom Vector Logo Component:** Engineered a pure vector SVG logo component featuring an emerald shopping bag embedded with an upward-trending growth chart line and data points. Also exported as the browser favicon.
- **Minimalist Clean Footer:** Streamlined footer to a single minimal line with copyright and store identity, removing extraneous clutter.

8. Setup, Installation & Verification Guide

Prerequisites

• Node.js version 18.0.0 or higher • npm or yarn • MongoDB Atlas cluster or local MongoDB (v6.0+) • Git

Step-by-Step Installation

Step 1: Clone Repository

```
git clone https://github.com/DevicharanPrajapati/ShopifyAnalyticsDashboard.git
cd ShopifyAnalyticsDashboard
```

Step 2: Server Environment

Create server/.env with:

```
PORT=5000
MONGODB_URL=mongodb+srv://:@cluster0.mongodb.net/shopify_store_analytics
CLIENT_URL=http://localhost:5173
```

Step 3: Install & Seed

```
cd server && npm install
npm run seed # Populates 8 products, 138 orders in INR, 30 days traffic
```

Step 4: Launch Server

```
npm run dev # Runs Express API on http://localhost:5000 with nodemon
```

Step 5: Client Setup

```
cd ../client && npm install
npm run dev # Launches Vite development server on http://localhost:5173
```

Step 6: Production Build

```
npm run build # Compiles production-ready bundle into client/dist
```

Project Completion & Status

All features, endpoints, visualizations, responsive layouts, filter systems, calculations, and server optimizations have been verified and pushed to the main branch of the GitHub repository. The project is completely functional and ready for evaluation.

Author / Developer: Devicharan Prajapati

Project: Shopify Store Analytics Dashboard

Store Identity: Apex Retailers (Single Store Architecture)

Repository: <https://github.com/DevicharanPrajapati/ShopifyAnalyticsDashboard.git>

Status: All Requirements Completed, Verified & Documented